Chapter 15: Virtual Functions and Polymorphism

ENG3091

Khiem Nguyen

Contents

1 Base class pointers and references **2**
- 1.1 Our `Base` and `Derived` . . . 2
- 1.2 Pointers and references to derived objects . . . 3
- 1.3 A *base* reference to a derived object . . . 3
- 1.4 A base reference sees only the base part . . . 4
- 1.5 Why would we want that? . . . 4
- 1.6 Couldn't a template do it? . . . 4
- 1.7 Our animals . . . 5
- 1.8 One array per kind of animal . . . 6
- 1.9 One vector of `Animal` pointers? . . . 7

2 Virtual functions **7**
- 2.1 Virtual functions . . . 7
- 2.2 A first virtual function . . . 8
- 2.3 Virtual functions in a chain . . . 9
- 2.4 The animals, with a virtual `speak()` . . . 10
- 2.5 Now every animal speaks for itself . . . 11
- 2.6 Behind the scenes (read at home) . . . 11
- 2.7 Polymorphism . . . 12
- 2.8 The rules of overriding . . . 12

3 The `override` and `final` specifiers **13**
- 3.1 Overrides that don't override . . . 13
- 3.2 `override` turns the bug into an error . . . 14
- 3.3 The `final` specifier . . . 14

4 Slicing and virtual destructors **16**
- 4.1 Object slicing . . . 16
- 4.2 What slicing means in practice . . . 16
- 4.3 Deleting through a base pointer . . . 17
- 4.4 Virtual destructors . . . 17

5 Pure virtual functions and abstract classes **18**
- 5.1 Forgetting to override . . . 18
- 5.2 Pure virtual functions . . . 18
- 5.3 Forgetting now fails to compile . . . 19
- 5.4 …and remembering works . . . 20
- 5.5 Interface classes . . . 20
- 5.6 Putting it all together: shapes . . . 20
- 5.7 A circle (and a rectangle) . . . 21
- 5.8 Every shape, in one loop . . . 22

6 Wrapping up **22**
6.1 Summary 22
6.2 Keep learning 22

1 Base class pointers and references

1.1 Our `Base` and `Derived`

Listing 1 `base_derived.h`

```
#ifndef BASE_DERIVED_H
#define BASE_DERIVED_H

#include <string_view>

class Base {
  protected:
    int id{};

  public:
    Base(int id) : id{id} {}
    std::string_view get_name() const { return "Base"; } // NOT virtual
};

class Derived : public Base {
  private:
    double value{};

  public:
    Derived(double value, int id) : Base{id}, value{value} {}
    std::string_view get_name() const { return "Derived"; }
    double get_value() const { return value; }
};

#endif
```

Nothing new here: `Derived` inherits from `Base`, and has its own `get_name()` that hides `Base::get_name()` (Chapter 14).

1.2 Pointers and references to derived objects

Listing 2 `pointers_references_to_derived.cpp`

```cpp
#include "base_derived.h"
#include <iostream>

int main() {
    Derived derived{42.24, 42};
    Derived& ref{derived};
    Derived* ptr{&derived};
    std::cout << derived.get_name() << ' ' << derived.get_value() << '\n';
    std::cout << ref.get_name() << ' ' << ref.get_value() << '\n';
    std::cout << ptr->get_name() << ' ' << ptr->get_value() << '\n';
    return 0;
}
```

```
Derived 42.24
Derived 42.24
Derived 42.24
```

Still nothing new: a `Derived` reference or pointer behaves just like the `Derived` object itself.

1.3 A *base* reference to a derived object

Listing 3 `base_pointers_references_to_derived.cpp`

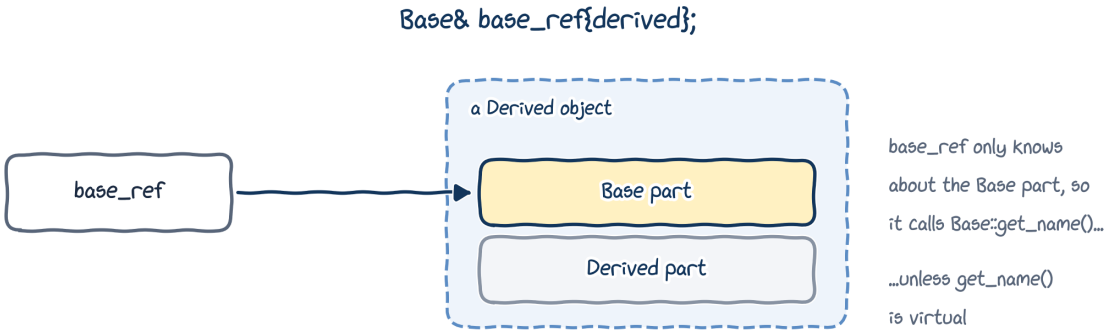
```cpp
#include "base_derived.h"
#include <iostream>

int main() {
    Derived derived{42.24, 42};
    Base& base_ref{derived};  // a Base reference to a Derived object
    Base* base_ptr{&derived}; // a Base pointer to a Derived object
    std::cout << derived.get_name() << '\n';
    std::cout << base_ref.get_name() << '\n';  // which get_name()?
    std::cout << base_ptr->get_name() << '\n'; // which get_name()?
    // base_ref.get_value(); // error: a Base has no get_value()
    return 0;
}
```

```
Derived
Base
Base
```

A `Derived` object **has a** `Base` part, so a `Base` reference or pointer can refer to it. But look at what it calls!

1.4 A base reference sees only the base part



`Derived::get_name()` hides `Base::get_name()` for `Derived` objects. But through a `Base` reference or pointer, the compiler only knows about the `Base` part, so it calls `Base::get_name()`.

1.5 Why would we want that?

Suppose we have cats, dogs and tigers, and want to report what each one says:

```
void report(const Cat& cat) {
    std::cout << cat.get_name() << " says " << cat.speak() << '\n';
}
void report(const Dog& dog) {
    std::cout << dog.get_name() << " says " << dog.speak() << '\n';
}
void report(const Tiger& tiger) {
    std::cout << tiger.get_name() << " says " << tiger.speak() << '\n';
}
```

A zoo with 100 kinds of animal means 100 copies of the same function. We'd much rather write one:

```
void report(const Animal& animal) {
    std::cout << animal.get_name() << " says " << animal.speak() << '\n';
}
```

1.6 Couldn't a template do it?

```
template <typename T>
void report(const T& animal) {
    std::cout << animal.get_name() << " says " << animal.speak() << '\n';
}
```

It works, but:

- nothing says `T` must be an animal: `report(42)` gives a long, confusing template error;
- each type gets its own instance, decided at **compile time**. A template can't handle a collection of *mixed* animals whose types are only known when the program runs.

1.7 Our animals

Listing 4 `animals_nonvirtual.h`

```
#ifndef ANIMALS_NONVIRTUAL_H
#define ANIMALS_NONVIRTUAL_H

#include <string>
#include <string_view>
class Animal {
  protected:
    std::string name{};
    Animal(std::string_view name) : name{name} {}

  public:
    std::string_view get_name() const { return name; }
    std::string_view speak() const { return "???"; }
};
class Cat : public Animal {
  public:
    Cat(std::string_view name) : Animal{name} {}
    std::string_view speak() const { return "Meow"; }
};
class Dog : public Animal {
  public:
    Dog(std::string_view name) : Animal{name} {}
    std::string_view speak() const { return "Woof"; }
};
#endif
```

The constructor of `Animal` is `protected`, so nobody can create a plain `Animal`: only cats and dogs.

`std::string_view` is a cheap, read-only view of some text (Chapter 04).

1.8 One array per kind of animal

Listing 5 `different_animal_arrays.cpp`

```cpp
#include "animals_nonvirtual.h"
#include <iostream>

int main() {
    // Without a common type, every kind of animal needs its own array...
    const Cat cats[]{Cat{"Fred"}, Cat{"Misty"}, Cat{"Zeke"}};
    const Dog dogs[]{Dog{"Garbo"}, Dog{"Pooky"}, Dog{"Truffle"}};

    for (const auto& cat : cats) { // ...and its own loop
        std::cout << cat.get_name() << " says " << cat.speak() << '\n';
    }
    for (const auto& dog : dogs) {
        std::cout << dog.get_name() << " says " << dog.speak() << '\n';
    }
    return 0;
}
```

```
Fred says Meow
Misty says Meow
Zeke says Meow
Garbo says Woof
Pooky says Woof
Truffle says Woof
```

An array holds objects of **one** type, so each kind of animal needs its own array and its own loop. With 100 kinds, that's 100 arrays.

1.9 One vector of `Animal` pointers?

Listing 6 `animal_array.cpp`

```cpp
#include "animals_nonvirtual.h"
#include <iostream>
#include <vector>

int main() {
    const Cat fred{"Fred"};
    const Cat misty{"Misty"};
    const Dog garbo{"Garbo"};
    const Dog pooky{"Pooky"};

    // One vector of Animal pointers holds cats AND dogs
    const std::vector<const Animal*> animals{&fred, &garbo, &misty, &pooky};

    for (const Animal* animal : animals) {
        std::cout << animal->get_name() << " says " << animal->speak() << '\n';
    }
    return 0;
}
```

```
Fred says ???
Garbo says ???
Misty says ???
Pooky says ???
```

Base pointers let one vector hold every kind of animal. But they all say `???`: each call goes to `Animal::speak()`. We need **virtual functions**.

2 Virtual functions

2.1 Virtual functions

A **virtual function** is a member function that, when called through a base pointer or reference, resolves to the **most-derived** version for the actual object.

- Put `virtual` in front of the function in the **base class**.
- A function in a derived class with the same signature and return type **overrides** it, and is called an **override**. Mark it `override`.

2.2 A first virtual function

Listing 7 `virtual_function_example.cpp`

```cpp
#include <iostream>
#include <string_view>

class Base {
  public:
    virtual std::string_view get_name() const { return "Base"; } // virtual!
};
class Derived : public Base {
  public:
    std::string_view get_name() const override { return "Derived"; }
};

int main() {
    Derived derived{};
    Base& base_ref{derived};
    std::cout << "base_ref is a " << base_ref.get_name() << '\n';
    return 0;
}
```

```
base_ref is a Derived
```

One word, `virtual`, and the call through `base_ref` now reaches `Derived::get_name()`.

2.3 Virtual functions in a chain

Listing 8 `virtual_function_inheritance_chain.cpp`

```cpp
#include <iostream>
#include <string_view>

class A {
  public:
    virtual std::string_view get_name() const { return "A"; }
};
class B : public A {
  public:
    std::string_view get_name() const override { return "B"; }
};
class C : public B {
  public:
    std::string_view get_name() const override { return "C"; }
};
class D : public C {
  public:
    std::string_view get_name() const override { return "D"; }
};

int main() {
    C c{};
    A& base_ref{c};
    std::cout << "base_ref is a " << base_ref.get_name() << '\n';
    return 0;
}
```

```
base_ref is a C
```

`base_ref` refers to a `C`, so `C::get_name()` is called: the most-derived version **for the actual object**. `D` doesn't come into it, because the object isn't a `D`.

2.4 The animals, with a virtual `speak()`

Listing 9 `animals.h`

```
#ifndef ANIMALS_H
#define ANIMALS_H

#include <string>
#include <string_view>
class Animal {
  protected:
    std::string name{};
    Animal(std::string_view name) : name{name} {}

  public:
    virtual ~Animal() = default; // a virtual destructor: see later
    std::string_view get_name() const { return name; }
    virtual std::string_view speak() const { return "???"; } // virtual!
};
class Cat : public Animal {
  public:
    Cat(std::string_view name) : Animal{name} {}
    std::string_view speak() const override { return "Meow"; }
};
class Dog : public Animal {
  public:
    Dog(std::string_view name) : Animal{name} {}
    std::string_view speak() const override { return "Woof"; }
};
#endif
```

Two changes from before:

- `virtual` in front of `Animal::speak()`;
- `override` after each derived `speak()`.

(And a virtual destructor, explained later today.)

2.5 Now every animal speaks for itself

Listing 10 `animals_with_virtual.cpp`

```cpp
#include "animals.h" // the same classes, with a virtual speak()
#include <iostream>
#include <vector>

void report(const Animal& animal) {
    std::cout << animal.get_name() << " says " << animal.speak() << '\n';
}

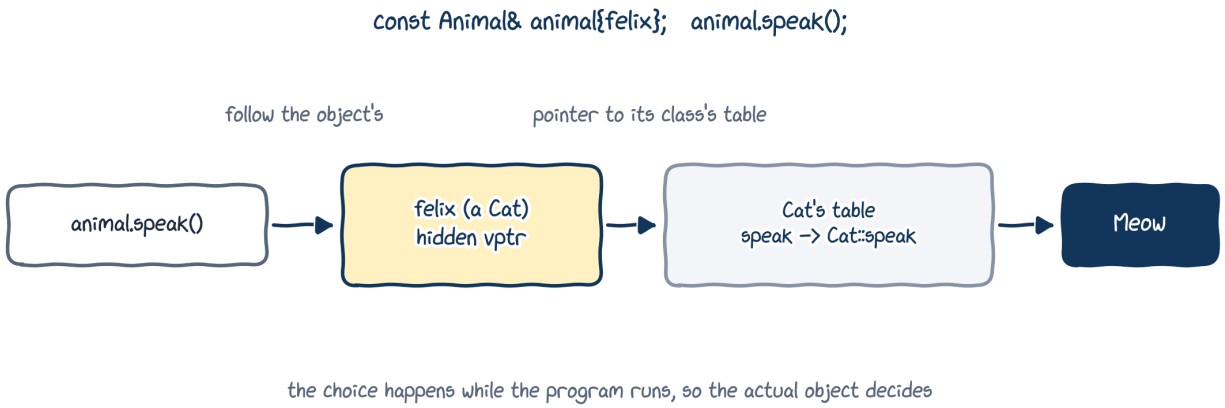
int main() {
    report(Cat{"Felix"});
    report(Dog{"Rex"});

    const Cat fred{"Fred"};
    const Dog garbo{"Garbo"};
    const std::vector<const Animal*> animals{&fred, &garbo};
    for (const Animal* animal : animals) {
        report(*animal);
    }
    return 0;
}
```

```
Felix says Meow
Rex says Woof
Fred says Meow
Garbo says Woof
```

The same `report()` function as before (`animals_without_virtual.cpp`, where everyone said `???`). Only the header changed.

2.6 Behind the scenes (read at home)



Every object of a class with virtual functions carries a hidden pointer to its class's **virtual table**, which lists the right function for each virtual call. Following it costs a tiny bit of time, which is why C++ functions aren't virtual unless you ask.

2.7 Polymorphism

Polymorphism means one name with many forms, chosen according to the types involved:

- **Compile-time polymorphism** is resolved by the compiler: function overloading (Chapter 05) and templates (Chapter 06).
- **Run-time polymorphism** is resolved while the program runs: **virtual functions**.

> **!** Key idea
>
> With virtual functions, code written for the **base class** (like `report(const Animal&)`) automatically does the right thing for **every** derived class, even ones written years later.

2.8 The rules of overriding

- An override needs **exactly** the same signature as the base function (parameter types and `const`), and is itself virtual.
- The return types must match too:

Listing 11 `virtual_return_type_mismatch.cpp`

```cpp
class Base {
  public:
    virtual int get_value() const { return 5; }
};
class Derived : public Base {
  public:
    double get_value() const { return 6.78; } // error: different return type
};

int main() {
    Derived derived{};
    return 0;
}
```

```
error: conflicting return type specified for 'virtual double Derived::get_value() const'
```

3 The `override` and `final` specifiers

3.1 Overrides that don't override

Listing 12 `not_use_override_specifier.cpp`

```cpp
#include <iostream>
#include <string_view>

class A {
  public:
    virtual std::string_view get_name_1(int) const { return "A"; }
    virtual std::string_view get_name_2(int) const { return "A"; }
};
class B : public A {
  public:
    // Look closely: neither of these overrides anything!
    virtual std::string_view get_name_1(short) const { return "B"; }
    virtual std::string_view get_name_2(int) { return "B"; } // not const
};

int main() {
    B b{};
    A& base_ref{b};
    std::cout << base_ref.get_name_1(1) << '\n';
    std::cout << base_ref.get_name_2(2) << '\n';
    return 0;
}
```

```
A
A
```

Both calls go to `A`. `B::get_name_1(short)` takes a `short`, not an `int`, and `B::get_name_2` isn't `const`: neither is an override. Only `-Wall` hints at it: *"… was hidden"*.

3.2 `override` turns the bug into an error

Listing 13 `use_override_specifier.cpp`

```cpp
#include <string_view>
class A {
  public:
    virtual std::string_view get_name_1(int) const { return "A"; }
    virtual std::string_view get_name_2(int) const { return "A"; }
    virtual std::string_view get_name_3(int) const { return "A"; }
};
class B : public A {
  public:
    std::string_view get_name_1(short) const override { return "B"; } // error
    std::string_view get_name_2(int) override { return "B"; }         // error
    std::string_view get_name_3(int) const override { return "B"; }   // fine
};

int main() {
    B b{};
    return 0;
}
```

```
error: 'B::get_name_1(short int) const' marked 'override', but does not override
error: 'B::get_name_2(int)' marked 'override', but does not override
```

> Best practice
>
> Write `virtual` only in the base class, and `override` on **every** override.

3.3 The `final` specifier

`final` stops a function from being overridden any further:

Listing 14 `final_specifier_example.cpp`

```cpp
#include <string_view>
class A {
  public:
    virtual std::string_view get_name() const { return "A"; }
};
class B : public A {
  public:
    std::string_view get_name() const override final { return "B"; }
};
class C : public B {
  public:
    std::string_view get_name() const override { return "C"; } // error
};

int main() {
    C c{};
    return 0;
}
```

```
error: virtual function 'virtual std::string_view C::get_name() const' overriding final function
```

And `class B final : public A` would stop anyone deriving from `B` at all.

4 Slicing and virtual destructors

4.1 Object slicing

Listing 15 `object_slicing.cpp`

```cpp
#include <iostream>
#include <string_view>

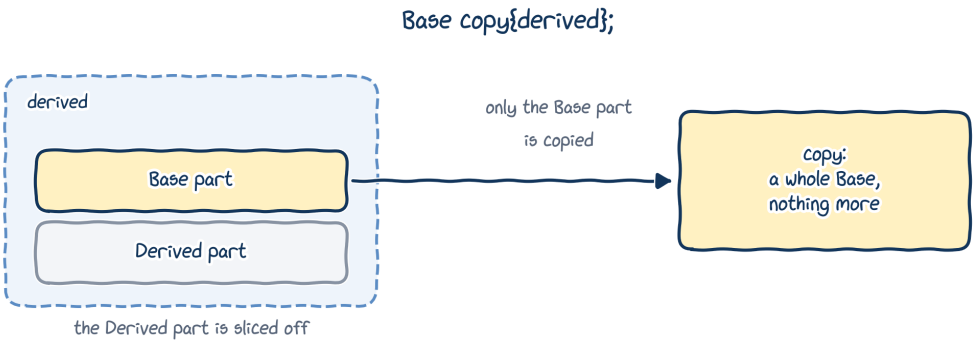
class Base {
  public:
    virtual ~Base() = default;
    virtual std::string_view get_name() const { return "Base"; }
};
class Derived : public Base {
  public:
    std::string_view get_name() const override { return "Derived"; }
};

int main() {
    Derived derived{};
    Base& ref{derived}; // a reference: refers to the whole Derived
    Base copy{derived}; // a copy: only the Base part is copied!
    std::cout << "ref is a " << ref.get_name() << '\n';
    std::cout << "copy is a " << copy.get_name() << '\n';
    return 0;
}
```

```
ref is a Derived
copy is a Base
```

A **reference** refers to the whole `Derived`. A **copy** into a `Base` keeps only the `Base` part: the rest is **sliced off**.

4.2 What slicing means in practice



- A `std::vector<Animal>` would slice every cat and dog into a plain `Animal`. That's why we used `std::vector<const Animal*>`.
- Pass polymorphic objects **by reference or pointer**, never by value.

4.3 Deleting through a base pointer

Listing 16 `virtual_destructor.cpp`

```cpp
#include <iostream>

class Base {
  public:
    ~Base() { std::cout << "~Base()\n"; } // NOT virtual
    virtual void hello() const { std::cout << "Hello from Base\n"; }
};
class Derived : public Base {
  public:
    ~Derived() { std::cout << "~Derived()\n"; } // cleans up Derived's part
};

int main() {
    Base* ptr{new Derived{}}; // a Base pointer to a heap Derived
    delete ptr;               // which destructors run?
    return 0;
}
```

```
$ g++ -Wall virtual_destructor.cpp
warning: deleting object of polymorphic class type 'Base' which has non-virtual destructor might caus
$ ./a.out
~Base()
```

`~Derived()` never ran! If `Derived` owned memory, it would leak.

4.4 Virtual destructors

Listing 17 `virtual_destructor_fixed.cpp`

```cpp
#include <iostream>

class Base {
  public:
    virtual ~Base() { std::cout << "~Base()\n"; } // virtual now
    virtual void hello() const { std::cout << "Hello from Base\n"; }
};
class Derived : public Base {
  public:
    ~Derived() override { std::cout << "~Derived()\n"; }
};

int main() {
    Base* ptr{new Derived{}};
    delete ptr;
    return 0;
}
```

```
~Derived()
~Base()
```

With a **virtual destructor**, `delete ptr` calls the destructor of the actual object, then the base destructors in turn (Chapter 14).

> Best practice
>
> A class meant to be a base class, especially one with virtual functions, should have a **virtual destructor**: `virtual ~Base() = default;`

5 Pure virtual functions and abstract classes

5.1 Forgetting to override

Listing 18 `virtual_function_forget_redefine.cpp`

```cpp
#include <iostream>
#include <string>
#include <string_view>

class Animal {
  protected:
    std::string name{};
    Animal(std::string_view name) : name{name} {}

  public:
    virtual ~Animal() = default;
    std::string_view get_name() const { return name; }
    virtual std::string_view speak() const { return "???"; }
};
class Cow : public Animal {
  public:
    Cow(std::string_view name) : Animal{name} {}
    // We forgot to override speak()!
};

int main() {
    Cow cow{"Betsy"};
    std::cout << cow.get_name() << " says " << cow.speak() << '\n';
    return 0;
}
```

```
Betsy says ???
```

We forgot `Cow::speak()`, so `Animal::speak()` quietly took over. No error, no warning: just a wrong answer.

5.2 Pure virtual functions

A **pure virtual function** has no body in the base class; it's marked `= 0`:

```cpp
virtual std::string_view speak() const = 0; // "every animal MUST say something"
```

- A class with at least one pure virtual function is an **abstract base class**: you **can't create objects** of it.
- A derived class must override every pure virtual function, or it's abstract too.
- Only virtual functions can be pure: `int do_something() = 0;` is a compile error.

So: what does an `Animal` say? There's no sensible default, and now we don't have to invent one.

5.3 Forgetting now fails to compile

Listing 19 `pure_virtual_function_forget_redefine.cpp`

```cpp
#include <string>
#include <string_view>
class Animal { // an abstract base class
  protected:
    std::string name{};
    Animal(std::string_view name) : name{name} {}

  public:
    virtual ~Animal() = default;
    std::string_view get_name() const { return name; }
    virtual std::string_view speak() const = 0; // pure virtual
};
class Cow : public Animal {
  public:
    Cow(std::string_view name) : Animal{name} {}
    // We forgot to override speak()... again!
};

int main() {
    Cow cow{"Betsy"}; // error: Cow is abstract too
    return 0;
}
```

```
error: cannot declare variable 'cow' to be of abstract type 'Cow'
```

A silent wrong answer is now a compile error: much better!

5.4 ...and remembering works

Listing 20 `pure_virtual_function_remember_redefine.cpp`

```cpp
#include <iostream>
#include <string>
#include <string_view>

class Animal { // an abstract base class
  protected:
    std::string name{};
    Animal(std::string_view name) : name{name} {}

  public:
    virtual ~Animal() = default;
    std::string_view get_name() const { return name; }
    virtual std::string_view speak() const = 0; // pure virtual
};
class Cow : public Animal {
  public:
    Cow(std::string_view name) : Animal{name} {}
    std::string_view speak() const override { return "Moo"; } // now we must
};

int main() {
    Cow cow{"Betsy"};
    std::cout << cow.get_name() << " says " << cow.speak() << '\n';
    return 0;
}
```

```
Betsy says Moo
```

5.5 Interface classes

An **interface class** has no member variables, and only pure virtual functions. It defines **what** derived classes must be able to do, and leaves **how** entirely to them. Interface names often start with `I`:

```cpp
class IErrorLog {
  public:
    virtual ~IErrorLog() = default; // so deleting through IErrorLog* is safe
    virtual bool open_log(std::string_view filename) = 0;
    virtual bool write_error(std::string_view message) = 0;
    virtual bool close_log() = 0;
};
```

Code that writes errors only needs an `IErrorLog&`. Whether errors go to a file, the screen or a database is decided by whichever class implements the interface.

5.6 Putting it all together: shapes

The `Shape` promised in Chapter 10, one class per header (Chapter 14):

Listing 21 `shape.h`

```
#ifndef SHAPE_H
#define SHAPE_H

#include <string_view>

class Shape { // an abstract base class: what every shape must provide
  public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
    virtual std::string_view name() const = 0;
};

#endif
```

An abstract base class with a virtual destructor: it only says what every shape must be able to do.

5.7 A circle (and a rectangle)

Listing 22 `circle.h`

```
#ifndef CIRCLE_H
#define CIRCLE_H

#include "shape.h"
#include <numbers> // std::numbers::pi (Chapter 04)

class Circle : public Shape {
  private:
    double radius{};

  public:
    Circle(double radius) : radius{radius} {}
    double area() const override { return std::numbers::pi * radius * radius; }
    std::string_view name() const override { return "circle"; }
};

#endif
```

`rectangle.h` is the same idea, with a `width` and a `height`.

5.8 Every shape, in one loop

Listing 23 `shapes.cpp`

```cpp
#include "circle.h"
#include "rectangle.h"
#include <iostream>
#include <memory> // std::unique_ptr (Chapter 09)
#include <vector>

int main() {
    std::vector<std::unique_ptr<Shape>> shapes{};
    shapes.push_back(std::make_unique<Circle>(1.0));
    shapes.push_back(std::make_unique<Rectangle>(2.0, 3.0));

    for (const auto& shape : shapes) { // one loop, every kind of shape
        std::cout << shape->name() << ": area " << shape->area() << '\n';
    }
    return 0;
} // the unique_ptrs delete the shapes, through ~Shape(): virtual!
```

```
circle: area 3.14159
rectangle: area 6
```

Abstract base class, overrides, a virtual destructor, and `std::unique_ptr` (Chapter 09) doing the `delete`: everything from today, working together.

6 Wrapping up

6.1 Summary

- A **base pointer or reference** can refer to a derived object, but only sees the base part.
- A **virtual function** call goes to the most-derived override for the **actual** object: run-time polymorphism.
- Write `virtual` in the base class and **`override`** on every override; **`final`** stops further overriding.
- Copying a derived object into a base object **slices** it: pass polymorphic objects by reference or pointer.
- Give base classes a **virtual destructor**.
- A **pure virtual function** (`= 0`) makes a class **abstract**; derived classes must override it. An **interface** is a class of pure virtual functions only.

6.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Virtual functions and polymorphism
2. The `override` and `final` specifiers, and covariant return types
3. Virtual destructors
4. Object slicing
5. Pure virtual functions, abstract base classes and interface classes

Next up, Section 4: describing object-oriented designs with UML.